

Desafio Técnico — Sistema de Estoque

Contexto

Este desafio tem como objetivo avaliar sua capacidade de:

- Modelar um problema real
- Aplicar arquitetura em camadas (MVC)
- Implementar regras de negócio consistentes
- Organizar código de forma clara e sustentável

O foco não está apenas em fazer funcionar, mas em **como você estrutura e resolve o problema**.

Objetivo

Desenvolver uma API para controle de estoque, permitindo:

- Cadastro de produtos
 - Registro de entradas de estoque
 - Registro de saídas de estoque
 - Consulta de saldo e movimentações
-

Entendendo o domínio (IMPORTANTE)

Antes de implementar, é esperado que você compreenda o funcionamento básico de um controle de estoque.

Produto

Representa um item controlado em estoque.

Exemplos:

- Notebook
- Mouse

- Cabo de rede

Cada produto possui um **saldo atual em estoque**, que deve ser atualizado conforme as movimentações.

Entrada de Estoque

Uma entrada representa um **evento de recebimento de produtos**.

Exemplo:

"Recebemos mercadorias do fornecedor"

Esse evento pode conter **vários itens diferentes**.

Aqui existe um conceito importante:

Mestre–Detalhe

- A entrada é o **mestre (evento principal)**
- Os itens são o **detalhe (produtos e quantidades)**

Ou seja:

- Uma entrada pode ter vários itens
 - Um item pertence a apenas uma entrada
-

Saída de Estoque

A saída representa a **retirada de um produto do estoque**.

Exemplo:

"Entrega de material para um setor"

Diferente da entrada:

- A saída é **simples (um produto por vez)**
-

Regras de Negócio

Produto

-
- Deve possuir nome
 - Pode estar ativo ou inativo
 - Produto inativo não pode ser movimentado
-

Entrada

- O Meste deve possuir o **código sequencial**, a **data da entrada**, o **valor total** com base nos itens inseridos
 - O Detalhe deve possuir pelo menos um item informando o **produto**, **quantidade** e **valor**
 - Cada item deve ter quantidade maior que zero
 - Ao registrar uma entrada:
 - O estoque dos produtos deve ser incrementado
-

Saída

- Deve possuir o **código sequencial**, a **data de saída**, o **produto**, a **quantidade**, e o **valor total**
 - Quantidade deve ser maior que zero
 - Não permitir saída com estoque insuficiente
 - Ao registrar saída:
 - O estoque deve ser decrementado
-

Consistência

O saldo de um produto deve sempre refletir corretamente:

- Todas as entradas realizadas
 - Todas as saídas realizadas
-

Arquitetura Esperada

O projeto deve seguir uma arquitetura em camadas.

Estrutura sugerida

- Controller
 - Service
 - Repository
 - DTO
 - Model
-

Expectativa da API

A API deve permitir executar todas as funcionalidades descritas de forma clara e organizada.

Espera-se:

- Endpoints bem definidos
 - Estrutura consistente de requisições e respostas
 - Separação entre entrada de dados e regra de negócio
-

Funcionalidades esperadas

Produto

- Criar produto
- Listar produtos
- Consultar produtos por ID
- Alterar status
- Deletar

Entrada

- Criar entrada com múltiplos itens
- Listar entradas
- Consultar entrada por ID

- Deletar
- Atualizar

Saída

- Registrar saída de produto
- Listar saídas
- Consultar saídas por ID
- Deletar
- Atualizar

Relatório

- Consultar saldo de produtos
- Exibir resumo de movimentações

Frontend (Opcional)

Caso opte por implementar frontend:

Sugestão: React

Banco de dados

PostgreSQL

Uso de Inteligência Artificial

Este desafio deve ser desenvolvido **sem o uso de ferramentas de IA**:

- O foco é avaliar seu raciocínio técnico e aprendizado
-

Entregáveis

- Repositório Git
 - Script do banco de dados
 - README com instruções de execução
-

Prazo

Entrega do desafio até o dia 15/05/2026

Decisões Técnicas

Sinta-se livre para tomar decisões de implementação.

Caso faça escolhas diferentes do esperado, explique no README o motivo.

Observação Final

Não esperamos uma solução perfeita.

Esperamos ver:

- Seu raciocínio
- Suas decisões técnicas
- Sua organização